WEEK 8

Stream Processing: Semantics & Engines

Real-Time Stream Processing

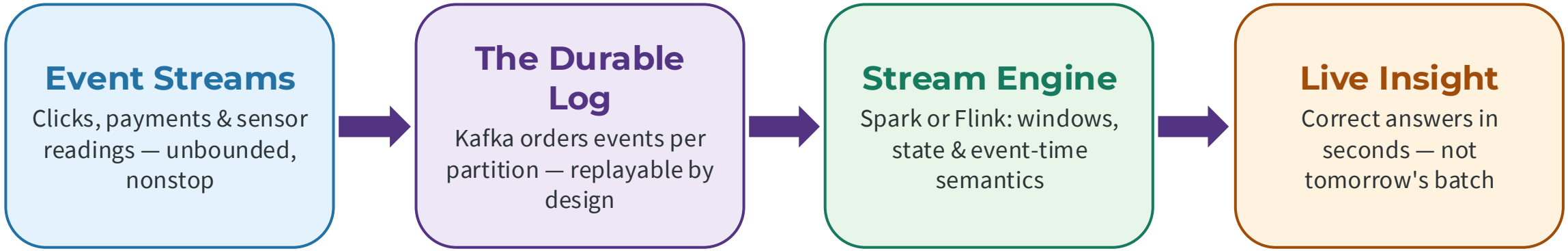
Time & Order
Event time, watermarks & late data

Windows & State
Tumbling, sliding & session windows

Exactly-Once
Guarantees · Spark SS vs Flink

CDLE • MEIC/MEIM • ISEL

From Events to Live Insight



This is the promise of stream processing

Process unbounded data with bounded resources — and deliver correct results, continuously, under explicit assumptions.

NEW TERMS

- `Kafka` durable, partitioned, replayable event log
- `Apache Flink` engine that processes each event on arrival
- `Window` finite slice of an infinite stream
- `State` what the engine remembers between events

Contents

Everything for week 8 — time, windows, state, exactly-once, and the engines that deliver them

01 Time & Order

Event time vs processing time
Out-of-order & late data
Watermarks: tracking event-time progress
Lateness policies & side outputs

02 Windows & State

Tumbling, sliding & session windows
Windowed aggregations in code
Stateful processing & its costs
Checkpointing & recovery

03 Exactly-Once & Engines

Delivery semantics: the three tiers
Exactly-once, end to end
Spark Structured Streaming vs **Flink**
Choosing an engine

04 Lab — Week 8

A windowed streaming analytics job
AI as adversary: corner cases
Your week-8 checklist
Next week: Lambda, Kappa & lakehouse

Scenario — Fraud, Right Now

Week 1 asked two questions — batch answered the first; now we answer the second

The Question

"Is fraud happening right now?" — same clickstream, but the answer must arrive in seconds

- Live clickstream: ≈50 000 events/s — unbounded, never complete
- A nightly batch answers **`~14 h late`** — the fraud has long cashed out
- Events arrive out of order — phones buffer, producers retry
- And the answer must still be correct, repeatable and auditable

The Napkin Math

```
# batch detection:
#   fraud at 12:04 → seen at 02:00
#   ≈ 14 h of exposure per incident

# streaming target: alert in < 30 s
the catch — someone must:
order events that arrive late,
  window time correctly,
  survive failures mid-stream,
  and count exactly once = hard
→ that is what this week is about
```

The lesson

Latency here is a correctness property. Streaming is not fast batch — it is a different semantics of time, order and state. This week we learn its rules.

NEW TERMS

- **`Clickstream`** user actions logged as a stream of events
- **`Producer`** application that writes events into Kafka
- **`Out-of-order`** arrives after events stamped later
- **`Auditable`** results can be traced and reproduced

From the Log to the Stream

Week 7 gave us a durable, replayable event log — this week we add computation on top of it

What Kafka Gave Us

- Topics & partitions — durable, ordered per partition
- Consumer groups — scalable, replayable reads
- Delivery guarantees — exactly-once within Kafka
- But a log stores events — it does not analyse them

What a Stream Engine Adds

- Continuous queries over unbounded streams
- Windows that slice infinite data into finite answers
- Managed state that survives failures
- Event-time semantics — correct results despite disorder

This Week's Pillars

Time | **Windows** | **State** | **Exactly-Once** | **Engines**

NEW TERMS

- **`Topic / partition`** named log, split into ordered shards
- **`Consumer group`** readers that share a topic's partitions
- **`Continuous query`** query that runs forever over new events
- **`Exactly-once`** each event affects the result once

Event vs Processing Time

Two clocks tick in every stream — when it happened, and when we finally see it

Event time — when it happened

```
# timestamp stamped at the source
event = {"card": "4929…",
   "amount": 950,
   "ts": "23:59:58"}   # truth
# immutable — travels with it
```

Processing — when we see it

```
# the wall clock of the engine
seen_at = now()
  # = "00:03:41"
  # 3 min 43 s after the fact
  # varies with queues & retries
```

The practical difference

```
event t.  = truth of the domain
proc. t.  = accident of arrival
the gap = lag, unbounded
analytics  = must use event time
```

✓ **Windows on event time**

✓ **Watermarks bound the wait**

✓ **Correct despite delays**

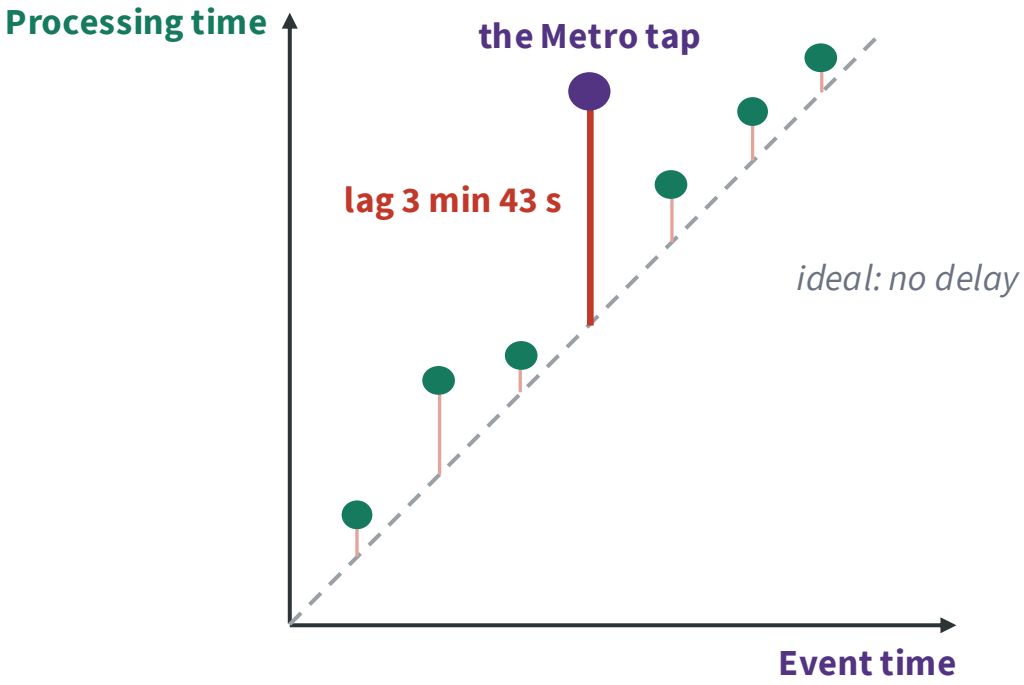
*The gap between the clocks is **lag** — it varies without bound; ordering by arrival lies about reality.*
***Every semantics this week** exists to manage that gap.*
A window closes once the watermark (newest event time − allowed delay) reaches its end; a late event never moves to another window.

NEW TERMS

- **`Event time`** when it happened, stamped at the source
- **`Processing time`** when the engine sees the event
- **`Lag`** gap between event and processing time
- **`Watermark`** engine's estimate of event-time progress

Two Clocks, One Event

Plot each event by when it happened against when the engine reads it: the vertical gap is lag



Event time (ET)

Stamped at the source; never changes on replay

Processing time (PT)

The engine's wall clock each time it reads the event

Lag = PT − ET

Differs per event: network, buffers
Changes on every replay

Build windows on **event time**: processing time changes every time you replay the log.
Metro tap: A card payment made on a phone with no signal on the Metro, stamped 23:59:58, seen at 00:03:41. Slide 8 follows it step by step.

NEW TERMS

- `Replay` re-reading events already stored in the log
- `Wall clock` the engine machine's current time

The Out-of-Order Reality

Follow one payment from a phone on the Metro — order is a luxury streams do not have

1 — *+1 min 14 s* → **2** — *+1 min 18 s* → **3** — *+1 min 11 s* → **4**

23:59:58 · The tap

A card is charged on a phone with no signal — the event is stamped and queued on the device.

00:01:12 · The buffer

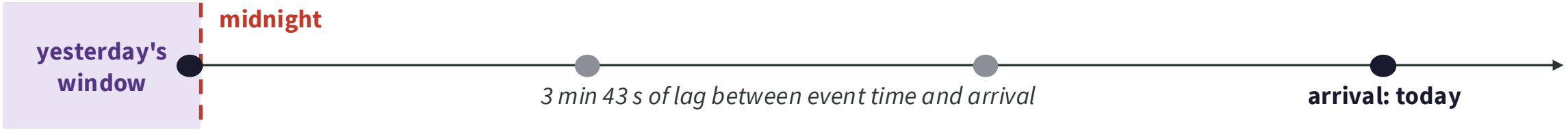
The phone regains signal and flushes its queue; thousands of devices do the same, unsynchronised.

00:02:30 · The retry

A leader change forces a producer retry — without idempotence, duplicates and reordering enter the log.

00:03:41 · The arrival

It arrives after midnight but belongs to yesterday's window — naïve arrival-order counts are already wrong.



Why this matters

- Lateness is normal, not exceptional: buffers, retries, partitions and clock skew guarantee disorder at scale.
- An engine that ignores it silently undercounts — watermarks (next slide) make lateness explicit and bounded.

NEW TERMS

- `Broker` Kafka server that stores partitions
- `Leader change` another replica takes over a partition
- `Duplicate` same event written to the log twice

Watermarks — The Rule

WM = max(ET seen) − allowed delay (here 10 min) · window [WST, WET) closes when WM ≥ WET

#	Event time	Processing time	Newest seen	Watermark	5-min window [23:55, 00:00)
1	23:57	23:58	23:57	**23:47**	counted · window open
2	00:02	00:03	00:02	**23:52**	event goes to next window · this one still open
3	**23:59:58 · Metro tap**	00:03:41	00:02	**23:52**	straggler, ahead of the mark → counted
4	00:11	00:12	00:11	**00:01**	**WM passes 00:00 → window closes with 2 events ***
5	23:58	00:14	00:11	**00:01**	behind the mark → late, policy decides (slide 12)

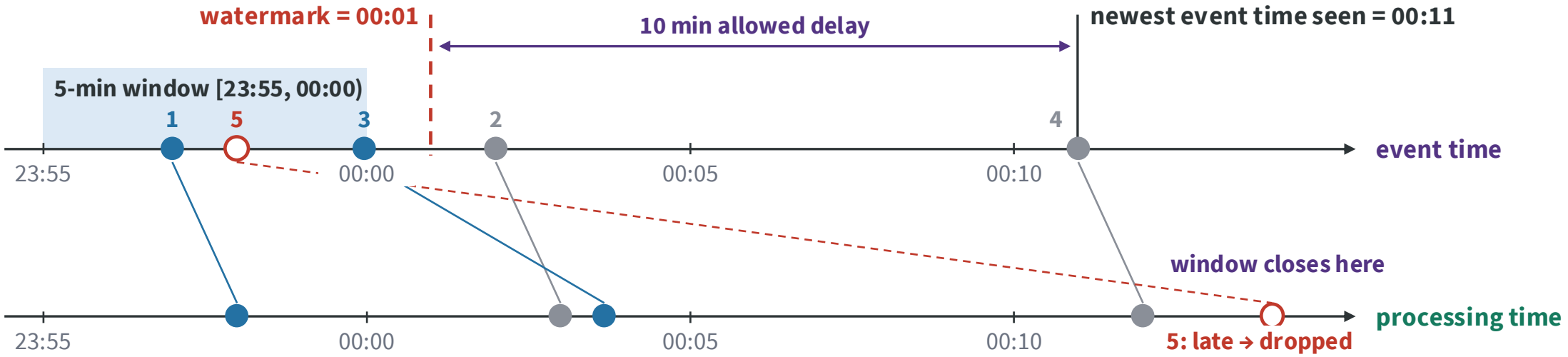
*An event counts if WST ≤ ET < WET and it arrives before WM ≥ WET — the watermark uses event time only. * event 4 itself belongs to a later window*

NEW TERMS

- `Watermark` (WM) engine's estimate of event-time progress
- `Late event` arrives behind the watermark
- `Straggler` delayed event still ahead of the mark
- `WST / WET` window start / end time

Watermark — Closing a Window

Slide 9's five events · each line joins an event's time (top) to its arrival (bottom) · 5-min windows · 10-min delay



1 · Before the mark
Events 1 and 3 count; WM is 23:52, so the window stays open

2 · Watermark passes 00:00
Event 4 arrives at PT 00:12: WM = 00:01, count of 2 emitted

3 · After the mark
Event 5 (ET 23:58) arrives at PT 00:14: too late, dropped

The watermark only advances with new data — if the stream goes quiet, open windows keep waiting.
Notes: event 3: ET = 23:59:58; Grey events: belong to later windows; Red and white event: dropped.

NEW TERMS

`Tumbling window` fixed-size windows with no overlap
`Final result` emitted once the mark passes the window

Watermarks in Spark Code

```
# how long do we wait for stragglers?
txn = parsed_events        # from Kafka
agg = (txn.withWatermark("ts", "10 minutes")
    .groupBy(window("ts", "5 minutes"))
    .agg(count("*").alias("n")))
# wm = max(event ts seen) - 10 min
# a window closes when wm ≥ its end

# the watermark bounds how long state
# is kept — and decides when a result
# may be declared final
```

What the Spark Job Does

- **withWatermark** → mark = newest seen − 10 min
- **window()** → 5-min tumbling windows
- **State** → kept until the mark passes
- **Update mode** → re-emits counts as they change
- **Append mode** → emits each window once, final
- **Late event** → behind the mark: may be dropped

A watermark is a bet, not a fact — one parameter trades latency against completeness. You will tune it tonight.

NEW TERMS

readStream Spark: open an unbounded streaming source
withWatermark Spark: event-time column + max delay
window() Spark: group rows into time buckets
Output mode which results each trigger writes out

Handling Late Data

When an event arrives behind the watermark, someone must decide its fate — the policy is yours, not the engine's

Within allowed lateness

```
windows stay open — state kept past the mark
updates — revised results replace old ones
downstream must tolerate corrections
  /* upserts, not blind appends */
cost — more state & longer retention
  /* memory grows with the grace period */
→ correctness first, latency second
```

Beyond the grace period

- Drop silently — the cheapest option, and invisibly wrong
- Side output (Flink) — route strays to a late-events channel
- Log & count — expose lateness as a monitored metric
- Reconcile in batch — the lakehouse fixes it (week 9)

One rule of thumb for every stream:

Measure | **Bound** | **Decide** | **Audit**

NEW TERMS

`Side output` extra stream for rejected or late events
`Upsert` insert, or update if the key exists
`Downstream` later stages that consume this output
`Lakehouse` data lake with transactional tables

Windowing — The Three Shapes

1 — Tumbling
fixed size, no overlap

2 — Sliding
fixed size, overlapping

3 — Session
gap-defined, data-driven

Three shapes, one question

```
# tumbling & sliding — clock-driven
tumbling ≈ every 5 min → one window per event
sliding (10 min, step 2) → 5 windows per event
  use: billing & SLAs vs trend detection
  → cost grows with the overlap

# session — defined by the data
gap ≈ 30 min of silence closes a session
length varies per user & visit — unknown upfront
  needs: per-key state, closed by the watermark
  → windows come from data, not clocks
```

Why the shape matters

- ✓ Tumbling: one answer per period — billing, SLA counts
- ✓ Sliding: smooth trends — anomaly & fraud detection
- ✓ Session: user behaviour — funnels, engagement, carts
- ✓ Every window is state the engine must keep and expire
- ✓ The watermark decides when a window may close

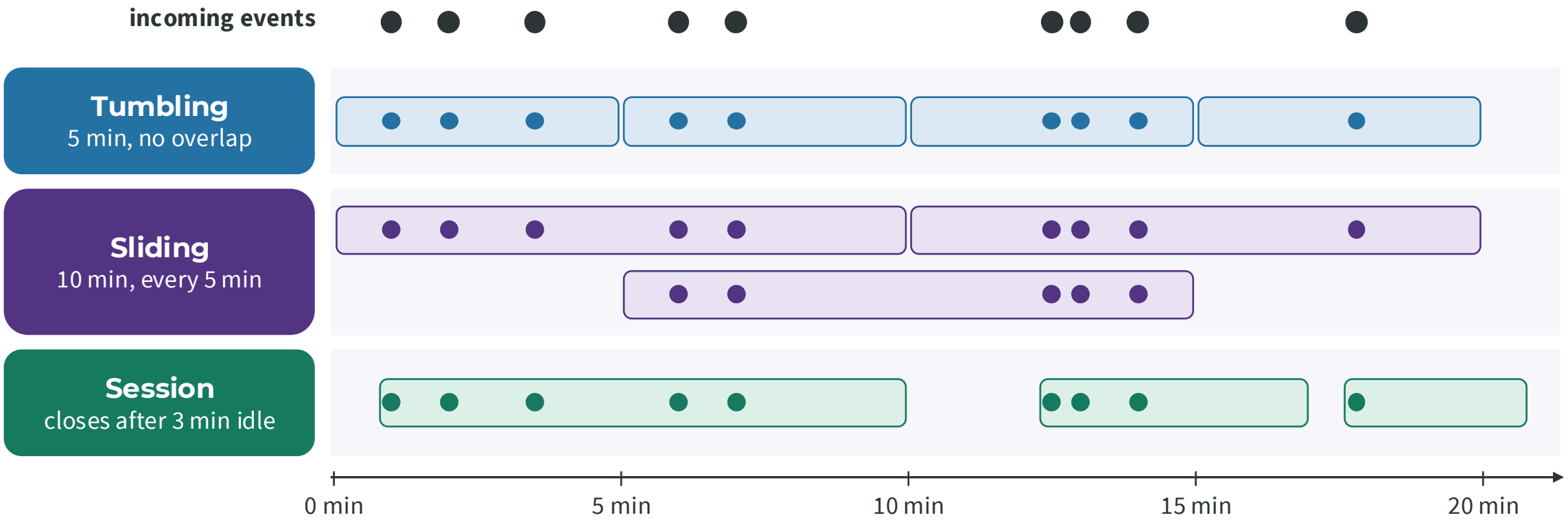
“Choose the window from the question, not from the engine's defaults — each shape answers a different query about time.”

NEW TERMS

`Sliding window` fixed size, overlapping, moves by a step
`Session window` closes after a gap of inactivity
`SLA` (Service Level Agreement) agreed service target, e.g. max latency

Three Windows, One Stream

The same payments for one user, sliced three ways along event time



Same nine events, three answers: tumbling counts each event once, sliding counts most events twice, sessions follow the user's rhythm.

Windows in Spark Code — The Job

window_job.py — the lab's job

```
events = kafka "trips1", parsed; + borough
agg = (events
  .withWatermark("pickup", "60 minutes") # lateness bet
  .groupBy(window("pickup", "5 minutes"), # tumbling
           col("borough"))                # keyed
  .agg(count("*"), sum("total")))

query = (agg.writeStream
  .outputMode("update")          # changed windows
  .option("maxOffsetsPerTrigger", 1000) # per batch
  .option("checkpointLocation", ckpt) # offsets+state
  .foreachBatch(upsert).start()) # MERGE into SQLite
```

What to notice

✓ **Watermark before groupBy**
Without it, window state grows forever — 10 minutes is your completeness bet

✓ **Output mode = update**
Open windows re-emit corrected counts as late events land

✓ **The checkpoint is the job**
Kill it, restart it — it resumes from offsets + state; the upsert makes replays harmless

This is the lab's job.
Note: for sliding window: window("pickup", "10 minutes", "2 minutes") # 10-min windows, every 2 min

NEW TERMS

Trigger how often the streaming query runs
Checkpoint saved state + offsets used to restart
Keyed separate result per key, e.g. borough

State — The Engine's Memory

✓ What state makes possible

- **Windows** — counts held open until the watermark closes them
- **Joins** — stream-stream & stream-table matching over time
- **Sessions** — per-key activity tracked across idle gaps
- **Dedup** — seen-before checks on event IDs
- **Patterns** — fraud sequences spanning many events (CEP)

	Stateless map, filter, route	Naïve state a dict in the process	Managed state store + checkpoints
POWER	✗ Limited	✓ Full	✓ Full
SAFE	✓ Nothing to lose	✗ Lost on crash	✓ Restored
COST	✓ Minimal	✗ Unbounded	✓ Bounded

State is streaming's superpower — and its liability: it must survive failures unharmed.

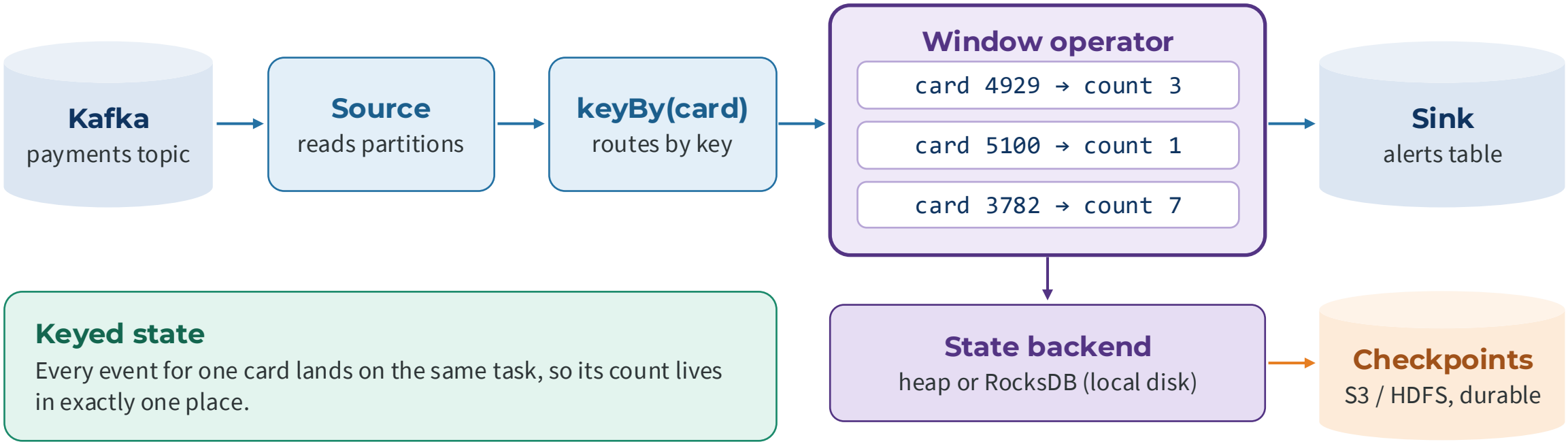
NEW TERMS

- `Stateless` each event handled alone, nothing kept
- `Join` matching events from two inputs
- `Dedup` dropping events already seen
- `CEP` Complex Event Processing: multi-event patterns

Where State Lives

Each key gets its own slice of state, kept beside the operator and snapshotted to durable storage

Flink terms



State is **partitioned by key**: scale out by spreading keys across tasks, recover by reloading the last snapshot.

NEW TERMS

- `Operator` one processing step in the dataflow
- `keyBy` routes all events of one key together
- `Sink` where the results are written
- `RocksDB` embedded key-value store on local disk

Checkpoints — Surviving Failure

State must outlive the process — every serious engine runs the same snapshot loop

1. Snapshot
Persist state + offsets, consistently

→

2. Fail
A node dies mid-window — it will happen

→

3. Restore
Restart from the last good snapshot

→

4. Replay
Re-read the log from the saved offsets

checkpoint/ — what actually gets saved

```
offsets: kafka
positions: <per partition>
state: window & key aggregates
commit: atomic (state ↔ offsets)
guarantee: "restart ≡ never crashed"
interval: seconds — bounds the replay
// Flink: async barrier snapshots
```

Failure modes to avoid

- **No checkpoint** — a restart reprocesses everything, or loses it all
- **Local-disk snapshots** — they die with the machine; use durable storage
- **Huge state, rare snapshots** — recovery takes as long as the outage
- **Changing the query** — old snapshots may not match the new plan

NEW TERMS

`Atomic commit` all-or-nothing write, never half-applied

`Barrier snapshot` Flink: async snapshot aligned by markers

`Offset` an event's position within a partition

Delivery — Three Promises

AT-MOST-ONCE	AT-LEAST-ONCE	EXACTLY-ONCE	IN PRACTICE
Fire & forget No retries — fast and cheap, but events vanish on any failure	**Retry till acked** Nothing is lost — but every retry can create duplicates	**The gold standard** Every event counted once — needs engine & sink cooperation	**Effectively once** Retries plus dedup or transactions make the effect exact

REMEMBER	THE COST	KAFKA'S PART	PAYOFF
It's end-to-end One leaky stage downgrades the whole pipeline's promise	**Coordination** Transactions & barriers add latency — pay only where needed	**Week 7 delivers** Idempotent producers & transactions do the transport half	**Auditable counts** Money, fraud & billing pipelines demand nothing less

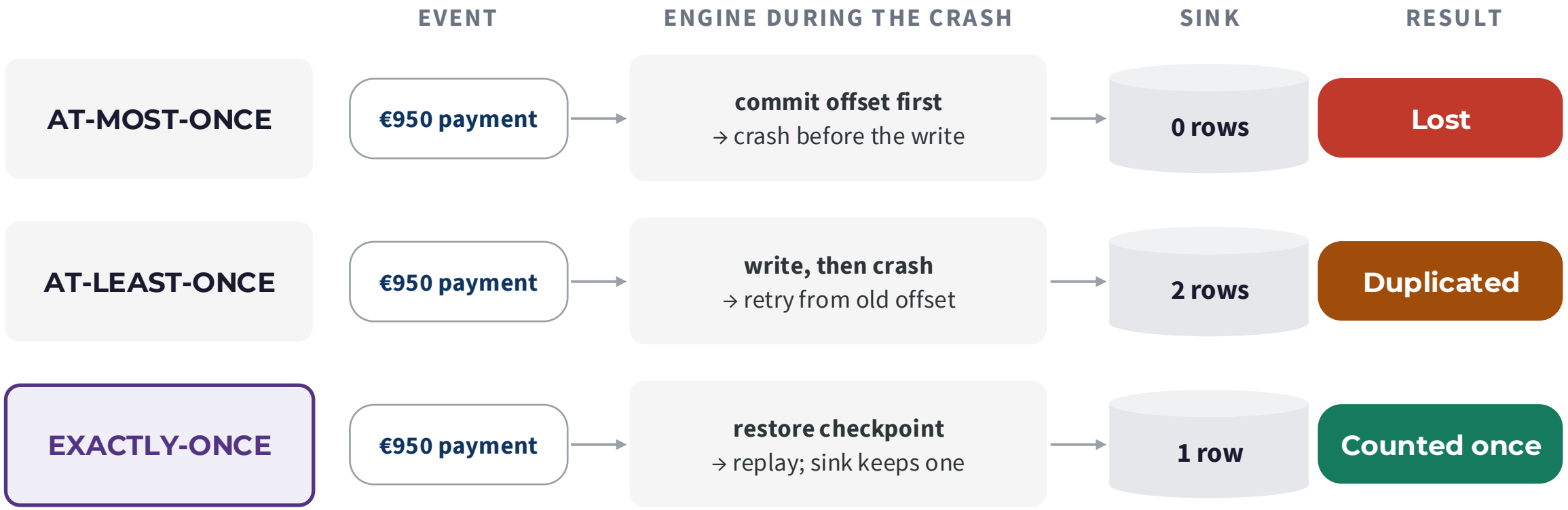
Choose the weakest promise your use case tolerates — metrics forgive duplicates; payments do not.
Exactly-once is engineered, not enabled.

NEW TERMS

- `At-most-once` no retries: failures lose events
- `At-least-once` retries: failures create duplicates
- `Idempotent producer` Kafka retries that never duplicate
- `Transaction` writes committed all-or-nothing

Three Promises Under Failure

Same payment, same crash: the delivery promise decides what reaches the sink



Exactly-once is **at-least-once plus deduplication**: replay everything, apply each effect once.

Exactly-Once, End to End

THE RECIPE

Three cooperating parts

no single flag — a contract across source, engine & sink

- **Replayable source** Kafka retains the log — after failure the engine re-reads from saved offsets
- **Checkpointed state** snapshots let reprocessing start from a consistent cut, not from zero
- **Transactional sink** outputs commit atomically with offsets — or land idempotently as upserts

THE FINE PRINT

What it does not mean

the phrase over-promises — know what you actually get

- **Effects, not deliveries** an event may be processed twice — its effect is applied exactly once
- **Ends matter** one sink neither transactional nor idempotent downgrades you to at-least-once
- **Scope is the pipeline** external calls (e-mails, APIs) escape the transaction — design for it

Golden rule: ***exactly-once is a property you engineer, not a flag you enable.***

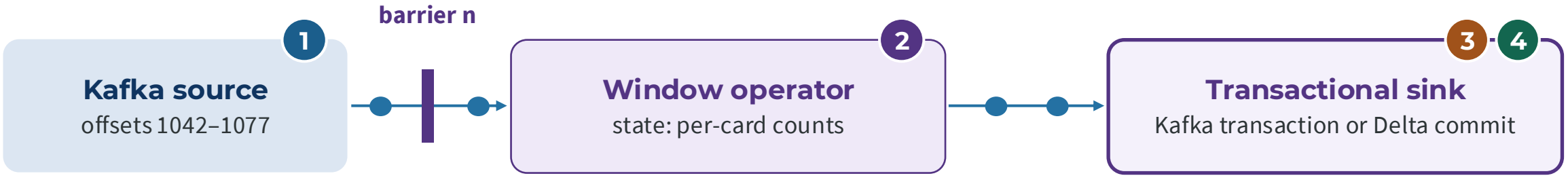
Spark SS and Flink both deliver it — but only when source, checkpoint and sink all hold their side of the contract.

NEW TERMS

- `Transactional sink` commits output with the offsets
- `Idempotent write` repeating it leaves the same result
- `Consistent cut` snapshot of all operators at one point
- `Spark SS` Spark Structured Streaming engine

The Exactly-Once Handshake

A checkpoint barrier flows with the data; the sink commits only when every operator has snapshotted



1 · Inject
The coordinator inserts barrier n at the source, after offset 1077

2 · Snapshot
Each operator saves its state and offsets as the barrier passes

3 · Pre-commit
The sink flushes writes into an open, invisible transaction

4 · Commit
Checkpoint n completes; the sink commits and results appear

Readers only ever see **committed** results: a crash before step 4 rolls the transaction back and replays from checkpoint n−1.

NEW TERMS

- `Checkpoint barrier` marker that triggers each snapshot
- `Coordinator` starts and completes each checkpoint
- `Pre-commit` writes staged, invisible until commit
- `Delta Lake` transactional table format on object storage

The Engine Landscape

Five ways to run a stream

Spark Struct. Streaming
micro-batch on the Spark stack

Apache Flink
event-at-a-time, low latency

Kafka Streams
a library, not a cluster

ksqlDB
streaming SQL over Kafka

Cloud managed
Dataflow · Managed Flink · Fabric RTI

How to place them



Latency
Flink & Kafka Streams react per event; Spark SS ticks in micro-batches



Weight
Libraries embed in your app; Spark & Flink run as clusters



Skills
Spark SS reuses everything you built in weeks 4–6



Reach
Managed services trade control for zero operations (week 10)

`This course works Spark SS & Flink` — *the two engines behind most serious production streaming. Slides 24–27 compare them.*

NEW TERMS

- `Micro-batch` small batch of recent events per trigger
- `Kafka Streams` Java library: streaming inside your app
- `ksqlDB` streaming SQL engine over Kafka topics
- `Managed service` the cloud runs, scales and patches it

Spark Structured Streaming

```
# a batch query that never ends
stream = spark.readStream.format("kafka")
agg = (stream.withWatermark("ts", "10 minutes")
    .groupBy(window("ts", "5 minutes"),
"country")
    .agg(sum("amount").alias("eur")))

(agg.writeStream.outputMode("update")
    .trigger(processingTime="30 seconds"))
# each trigger runs a small Spark job —
# Catalyst optimises it — weeks 4–6
```

The Model

- **Micro-batch** → seconds of latency, huge throughput
- **Unified API** → the same DataFrame code as batch
- **Triggers** → decide when each micro-batch fires
- **Output modes** → append · update · complete
- **Checkpoint** → offsets + state; exactly-once with a transactional sink
- **Real-Time Mode** → 4.1+, ms latency, stateless only

If you know Spark, you know 80% of this — streaming becomes an incremental view over the engine you already use.

NEW TERMS

Catalyst Spark's query optimiser
Complete mode rewrites the whole result each trigger
Real-Time Mode Spark 4.1+ low-latency trigger

Apache Flink — The Purist

fraud_score.py — PyFlink, per-card sessions

```
env = StreamExecutionEnvironment.get_execution_environment()
events = env.from_source(kafka_src, wm_strategy, "payments")
scored = (events
  .key_by(lambda e: e.card)       # per card
  .window(EventTimeSessionWindows
          .with_gap(Time.minutes(30)))  # sessions
  .aggregate(FraudScore())
  .filter(lambda s: s.score > THRESHOLD))
scored.sink_to(alerts_sink)   # on session close
# runtime: one long-lived graph of operators,
#   keyed state local to each operator,
#   barrier snapshots, no micro-batches
```

What to notice

✓ **Events flow one at a time**
Operators react in ms — but a window emits only when it fires

✓ **State lives with the operator**
RocksDB keyed state + async barrier snapshots give exactly-once

✓ **Time is a first-class citizen**
Watermarks, timers & session windows are native primitives

Flink is the reference implementation of the Dataflow model — when milliseconds or huge state bite, it is the answer.

NEW TERMS

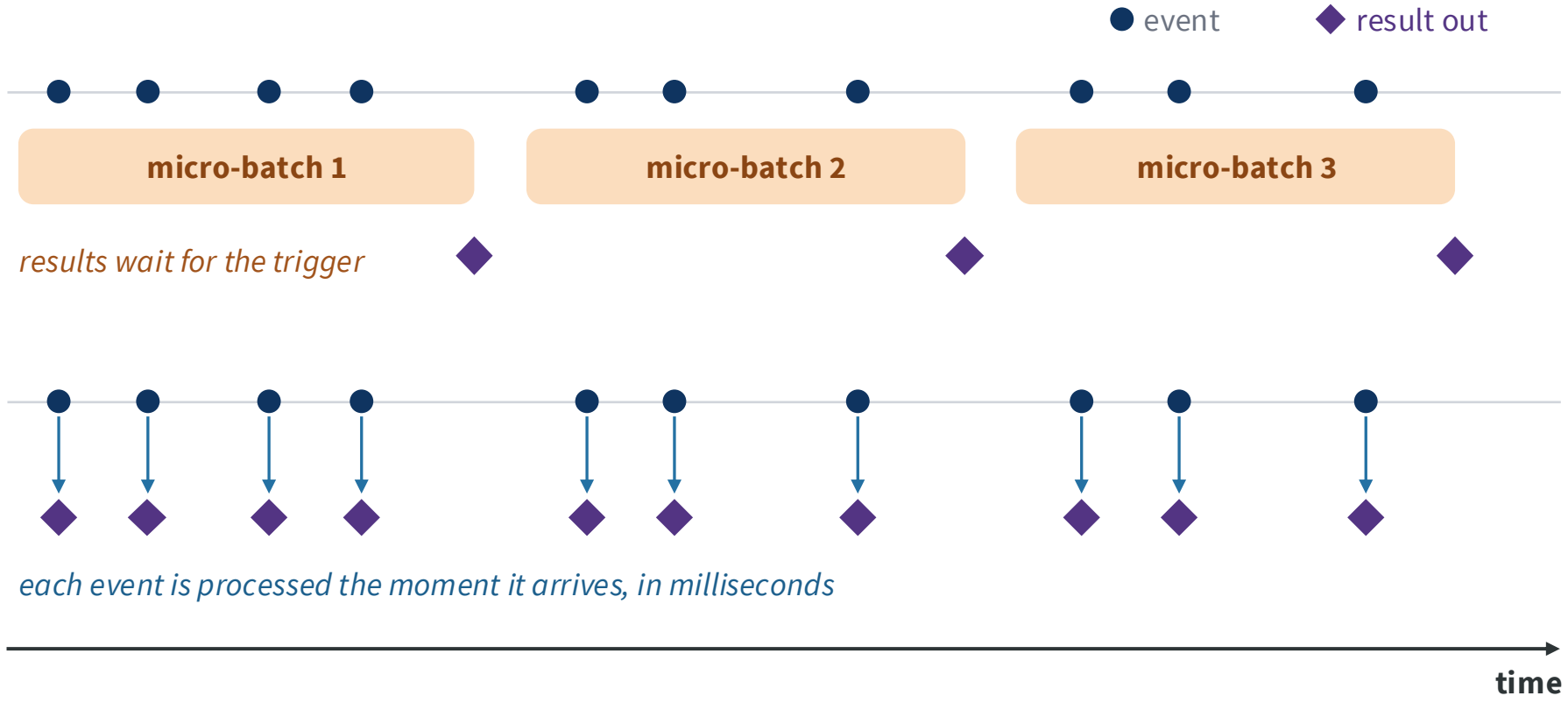
- **PyFlink** Flink's Python API
- **StreamExecutionEnv.** entry point of a Flink job
- **wm_strategy** Flink rule for generating watermarks
- **Event-at-a-time** each event processed as it arrives

Micro-batch vs Event-at-a-Time

Spark gathers events into small batches; **Flink** pushes each event through the moment it arrives

Spark Structured Streaming
latency ≈ 100 ms to secs

Apache Flink
latency ≈ ms



Same final results for event-time queries; **micro-batch trades latency for throughput**, event-at-a-time reacts in milliseconds.

Choosing — Spark SS vs Flink

Dimension	Spark Structured Streaming	Apache Flink	Edge
Latency	`Micro-batch: ~100 ms – s`	Milliseconds — per event	Flink (stateful)
Throughput	`Excellent, batch-optimised`	Excellent, sustained	Tie
Semantics	`Exactly-once via checkpoints`	Exactly-once, barrier snapshots	Tie
State & timers	`transformWithState (4.0+)`	Fine-grained: timers, CEP	Flink
Ecosystem	`One engine: batch + ML + SQL`	Streaming-first, growing SQL	Spark
Team skills	`Weeks 4–6 carry over directly`	New APIs & operations to learn	Spark

`Default advice: stay with Spark SS when Spark is already your platform;`
`reach for Flink when milliseconds, huge state or event patterns rule.`

…both implement the same semantics you learnt today — time, watermarks, windows, state, exactly-once. Concepts transfer; only APIs differ.

NEW TERMS

`Timer` callback fired at a chosen time

Reading List

Ranked by priority — the primary set underpins the lab and next week; the secondary set goes deeper on the slides listed.

1 · Primary reading — read before the lab

1 **Akidau et al., *The Dataflow Model*, VLDB 2015**
Event time, windows, watermarks and triggers · Slides 6–13

2 **Akidau, Chernyak & Lax, *Streaming Systems*, 2018**
Ch. 2–3 watermarks & windows, ch. 5 exactly-once · Slides 9–22

3 **Apache Spark docs, *Structured Streaming Programming Guide***
Watermarks, output modes, checkpointing · Slides 11, 15, 24

4 **Kleppmann, *Designing Data-Intensive Applications*, 2017**
Ch. 11 stream processing; due before week 9 · Slides 16–22

2 · Secondary reading — go deeper

1 **Carbone et al., *Apache Flink***, IEEE Data Eng. Bull. 2015
Stream and batch processing in one engine · Slides 23, 25

2 **Carbone et al., *Lightweight Asynchronous Snapshots***, 2015
The barrier checkpoints behind Flink's exactly-once · Slides 18, 22

3 **Armbrust et al., *Structured Streaming***, SIGMOD 2018
Spark's declarative streaming API and micro-batches · Slides 24, 26

4 **Akidau, *Streaming 101 & 102***, O'Reilly, 2015
Plain-language intro to event time and windows · Slides 6–13

5 **Apache Flink docs, *Stateful Stream Processing***
Keyed state, state backends and checkpoints · Slides 16–18

6 **Apache Kafka, *KIP-98: Exactly-Once Semantics***, 2017
Idempotent producers and transactions · Slides 19–21

Read the primary set in order; pick secondary items as your lab job needs.

PART 2 · HANDS-ON

Lab

08